ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №2 (Яндекс.Контест)

Выполнил:

Студент группы P3213

Якименко Владислав Игореич

Преподаватель:

Сорокин Артём Николаевич

Санкт-Петербург, 2026г.

Задачи из 2-го блока Яндекс.Контеста

Задача «E. Коровы в стойла**»**

На прямой расположены стойла, в которые необходимо расставить коров так, чтобы минимальное расстояние между коровами было как можно больше.

Формат ввода

В первой строке вводятся числа NN (2<N≤105)(2<N≤105) – количество стойл и KK (1<K<N)(1<K<N) – количество коров. Во второй строке задаются N натуральных чисел в порядке возрастания – координаты стойл (координаты не превосходят 109109).

Формат вывода

Выведите одно число – наибольшее возможное допустимое расстояние.

Пример 1

Ввод	**Вывод**
6 3 2 5 7 11 15 20	9

Пример 2

Ввод	**Вывод**
5 3 1 2 3 100 1000	99

Идея решения:

Идея решения заключается в том, чтобы расставить k коров по стойлам так, чтобы минимальное расстояние между любыми двумя поставленными коровами было как можно больше. Это обычная задача на бинарный поиск по ответу. Будем искать ответ не по позиции, а по самому расстоянию. Пусть мы проверяем некоторое расстояние x: можно ли расставить коров так, чтобы между соседними стояло хотя бы x.

Для такой проверки используем жадный алгоритм (жадник):

- первую корову ставим в первое стойло
- дальше идём слева направо
- если текущее стойло находится от последнего занятого не ближе чем на x, ставим туда следующую корову

Если таким способом удалось поставить хотя бы k коров, значит расстояние x подходит

Если x подходит, пробуем увеличить расстояние.

Если x не подходит, уменьшаем его.

Так бинарным поиском находим максимальное допустимое расстояние.

Сложность:

- Время: O(n log R), где R - это разность между крайними стойлами (первым и последним)
- Память: O(n)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <iostream>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                  \
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

bool ok(vector<ll>& v, ll n, ll k, ll x) {
  ll cnt = 1;
  ll last = v[0];
  for (ll i = 1; i < n; ++i) {
    if (v[i] - last >= x) {
      ++cnt;
      last = v[i];
    }
  }
  return cnt >= k;
}

void solve() {
  ll n = 0, k = 0;
  cin >> n >> k;

  vector<ll> v(n);
  for (ll i = 0; i < n; ++i) {
    cin >> v[i];
  }

  ll l = 0;
  ll r = v[n - 1] - v[0] + 1;
  while (r - l > 1) {
```

```
    ll m = (l + r) / 2;
    if (ok(v, n, k, m)) {
      l = m;
    } else {
      r = m;
    }
  }

  cout << l << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача «F. Число»

Вася написал на длинной полоске бумаги большое число и решил похвастаться своему старшему брату Пете этим достижением. Но только он вышел из комнаты, чтобы позвать брата, как его сестра Катя вбежала в комнату и разрезала полоску бумаги на несколько частей. В результате на каждой части оказалось одна или несколько идущих подряд цифр.

Теперь Вася не может вспомнить, какое именно число он написал. Только помнит, что оно было очень большое. Чтобы утешить младшего брата, Петя решил выяснить, какое максимальное число могло быть написано на полоске бумаги перед разрезанием. Помогите ему!

Формат ввода

Входной файл содержит одну или более строк, каждая из которых содержит последовательность цифр. Количество строк во входном файле не превышает 100, каждая строка содержит от 1 до 100 цифр. Гарантируется, что хотя бы в одной строке первая цифра отлична от нуля.

Формат вывода

Выведите в выходной файл одну строку — максимальное число, которое могло быть написано на полоске перед разрезанием.

Пример 1

Ввод	**Вывод**
2 20 004	66220004

Ввод	**Вывод**
66	

Пример 2

Ввод	**Вывод**
3	3

Идея решения:

Задача такова, что требуется из нескольких кусков, на каждом из которых записана строка цифр, собрать максимально возможное число. Мы можем только менять порядок кусков, а внутри каждого куска цифры переставлять нельзя

Главная идея такая: для двух кусков x и y надо понять, в каком порядке их выгоднее поставить:

- если x + y > y + x, то x должно идти раньше y
- иначе раньше должно идти y

То есть мы сортируем все строки не по обычному порядку, а по специальному правилу сравнения: какая из двух склеек больше, тот кусок и ставим раньше. После такой сортировки просто склеиваем все строки в одну.

Сложность:

- Время: O(n log n)
- Память: O(n)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                 \
  do {                           \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);            \
    cout.tie(nullptr);           \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)
```

```
/*
 * HEADER END
 */

void solve() {
  vector<string> a;
  string s;
  while (cin >> s) {
    a.push_back(s);
  }

  sort(all(a), [](const string& x, const string& y) { return x + y > y + x; });

  string ans;
  for (ll i = 0; i < static_cast<ll>(a.size()); ++i) {
    ans += a[i];
  }

  cout << ans << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача «G. Кошмар в замке»

Ходят легенды, что пока Аврора спала, ей снилось, что она ходит по разным местам: леса, поля, города и сёла. И вот однажды она наткнулась на пещеру, в которой сидел мудрец. Когда мудрец поднял на Аврору глаза, он изрёк: «*Дорогая Аврора! Ты уже годами скитаешься по этим землям. Я хочу предложить тебе задачку. Вот тебе строка* s*. Каждая буква из алфавита имеет свой вес* c_i*. Вес строки, которую ты можешь получить из* s *многократным обменом любых двух букв, вычисляется так: для каждой буквы алфавита посчитай максимальное расстояние между позициями, в которых стоит эта буква и перемножь его с весом этой буквы. Принеси мне строку максимально возможного веса, и я тебе расскажу, в чём смысл жизни*».

К счастью, когда Аврора уже шла со строкой к мудрецу, её поцеловал Филипп, и Аврора вышла из этого кошмара. Теперь вам предлагается самим окунуться в этот кошмар и решить поставленную задачу.

Формат ввода

Дана строка, состоящая из строчных букв латинского алфавита ($1 \le |s| \le 105$)($1 \le |s| \le 105$). Следующая строка ввода содержит 26 чисел — веса букв латинского алфавита от «a» до «z», веса неотрицательны и не превосходят 231−1231−1.

Формат вывода

Выведите строку s*s*, в которой переставлены буквы так, чтобы полученный вес был максимально возможным. Если искомых вариантов несколько, выведите любой из них.

Пример

Ввод	**Вывод**
lkshrules 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	slkhruels

Идея решения:

Мы можем переставлять буквы строки как угодно, значит важно не исходное расположение букв, а только то, сколько раз каждая буква встречается, тогла для каждой буквы вклад в ответ считается так:

- вес буквы умножается на максимальное расстояние между самой левой и самой правой её позицией
- если буква встречается один раз или не встречается вообще, её вклад равен 0

Значит для букв, которые встречаются хотя бы 2 раза, выгодно поставить одну копию как можно левее, а другую как можно правее. Тогда расстояние для этой буквы будет максимальным.

Если таких букв несколько, они как бы занимают “слои” с краёв:

- одна буква ставится на самые внешние позиции
- другая на следующие и так далее.

Если буква занимает слой с номером r, то её расстояние будет равно n - 2r + 1

Чем ближе к краю стоит буква, тем больше её расстояние, значит тем выгоднее отдавать внешние слои буквам с большими весами

Присутствует ограничение в виде того, что если буква встречается cnt раз, то после того как мы поставили две её копии на некоторый слой, все остальные cnt - 2 копий должны уместиться между ним. Из этого получается ограничение на то, насколько далеко к центру можно отодвинуть пару этой буквы. Дальше задача сводится к жадному распределению слоёв (жадник):

- для каждой буквы, которая встречается хотя бы 2 раза, считаем максимальный допустимый слой
- сортируем такие буквы по этому ограничению

- идём от самых дальних слоёв к ближним
- на каждом шаге среди букв, которые уже можно поставить на этот слой, выбираем букву с максимальным весом
- выбранные буквы ставим попарно симметрично по краям
- все оставшиеся буквы дописываем в середину в любом порядке.

Сложность:

Время: O(n + 26 log 26), потому что алфавит фиксированный: только строчные латинские буквы от a до z. То есть всего возможных букв ровно 26, и все циклы/сортировки по буквам идут не по n, а именно по этим 26 символам:

- Память: O(n)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <algorithm>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                  \
  do {                            \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);             \
    cout.tie(nullptr);            \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  string s;
  cin >> s;

  vector<ll> w(26);
  for (ll i = 0; i < 26; ++i) {
    cin >> w[i];
  }

  ll n = static_cast<ll>(s.size());
  vector<ll> cnt(26, 0);
  for (ll i = 0; i < n; ++i) {
```

```
  ++cnt[s[i] - 'a'];
}

vector<vector<ll>> jobs;
for (ll i = 0; i < 26; ++i) {
  if (cnt[i] >= 2) {
    ll lim = (n - cnt[i] + 2) / 2;
    jobs.push_back({lim, w[i], i});
  }
}

sort(all(jobs), [](const vector<ll>& a, const vector<ll>& b) {
  if (a[0] == b[0]) {
    if (a[1] == b[1]) {
      return a[2] > b[2];
    }
    return a[1] > b[1];
  }
  return a[0] > b[0];
});

ll t = static_cast<ll>(jobs.size());
vector<ll> rk(26, 0);
priority_queue<pair<ll, ll>> q;
ll p = 0;
for (ll cur = t; cur >= 1; --cur) {
  while (p < static_cast<ll>(jobs.size()) && jobs[p][0] >= cur) {
    q.push({-jobs[p][1], jobs[p][2]});
    ++p;
  }
  pair<ll, ll> x = q.top();
  q.pop();
  rk[x.second] = cur;
}

string ans(n, '?');
vector<ll> left = cnt;
for (ll i = 0; i < 26; ++i) {
  if (rk[i] > 0) {
    ll r = rk[i];
    ans[r - 1] = static_cast<char>('a' + i);
    ans[n - r] = static_cast<char>('a' + i);
    left[i] -= 2;
  }
}

ll pos = t;
for (ll i = 0; i < 26; ++i) {
  while (left[i] > 0) {
    ans[pos] = static_cast<char>('a' + i);
    ++pos;
    --left[i];
  }
```

```
  }

  cout << ans << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```

Задача « Н. Магазин»

У Билла большая семья: трое сыновей, девять внуков. И всех надо кормить. Поэтому Билл раз в неделю ходит в магазин.

Однажды Билл пришел в магазин и увидел, что в магазине проводится акция под названием «каждый kk-й товар бесплатно». Изучив правила акции, Билл выяснил следующее. Пробив на кассе товары, покупатель получает чек. Пусть в чеке nn товаров, тогда n/kn/k округленное вниз самых дешевых из них достаются бесплатно.

Например, если в чеке пять товаров за 200, 100, 1000, 400 и 100 рублей, соответственно, и k=2k=2, то бесплатно достаются оба товара по 100 рублей, всего покупатель должен заплатить 1600 рублей.

Билл уже выбрал товары, и направился к кассе, когда сообразил, что товары, которые он хочет купить, можно разбить на несколько чеков, и благодаря этому потратить меньше денег.

Помогите Биллу выяснить, какую минимальную сумму он сможет заплатить за выбранные товары, возможно разбив их на несколько чеков.

Формат ввода

Первая строка входного файла содержит два целых числа nn, kk (1≤n≤1000001≤n≤100000, 2≤k≤1002≤k≤100) — количество товаров, которые хочет купит Билл и параметр акции «каждый kk-й товар бесплатно».

Следующая строка содержит nn целых чисел aiai (1≤ai≤100001≤ai≤10000) — цены товаров, которые покупает Билл.

Пример

Ввод	**Вывод**
5 2	

Ввод	**Вывод**
200 100 1000 400 100	

Идея решения:

Несложно заметить, что в одном чеке бесплатно даются floor(m / k) самых дешёвых товаров этого чека, где m является число товаров в чеке. Чтобы получить максимальную выгоду, нужно делать бесплатными как можно более дорогие товары. Для этого выгодно сначала отсортировать все цены. Если отсортировать товары по возрастанию, то оптимально рассматривать их с конца, то есть от самых дорогих к самым дешёвым. Тогда каждый k-й товар в таком порядке можно сделать бесплатным, а остальные нужно оплатить.

- сортируем все цены
- идём с конца массива к началу
- считаем товары по порядку
- если номер товара кратен k, то этот товар считаем бесплатным
- иначе прибавляем его цену к ответу

Сложность:

- Время: O(n log n)
- Память: O(n)

Код:

```
/**
 * @author: Slava Yakimenko
 */

#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;
typedef long long ll;

#define fastIO()                 \
  do {                           \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);            \
    cout.tie(nullptr);           \
  } while (0)
#define deb(x) cout << #x << " = " << x << '\n'
#define all(x) x.begin(), x.end()
#define el '\n'
#define INF static_cast<ll>(1e17)

/*
 * HEADER END
 */

void solve() {
  ll n = 0, k = 0;
```

```
  cin >> n >> k;

  vector<ll> a(n);
  for (ll i = 0; i < n; ++i) {
    cin >> a[i];
  }

  sort(all(a));
  reverse(all(a));

  ll ans = 0;
  for (ll i = 0; i < n; ++i) {
    if ((i + 1) % k != 0) {
      ans += a[i];
    }
  }

  cout << ans << el;
}

int main() {
  fastIO();
  // freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
  bool multitest = false;
  ll T = 1;
  if (multitest) {
    cin >> T;
  }
  for (ll i = 0; i < T; ++i) {
    solve();
  }
}
```